DATA VERSION CONTROL (DVC)

Practical Assignment Report

Project: DVC data versioning using Git

Dataset/File: data.txt

Versions demonstrated: v1.0 and v2.0

1. Objective

To demonstrate practical Data Version Control using a simple data file, while using Git to maintain the versioned DVC metadata. The practical covers initialization, creation of two data versions, committing the corresponding metadata, tagging versions, and restoring an earlier data version.

2. Tools Used

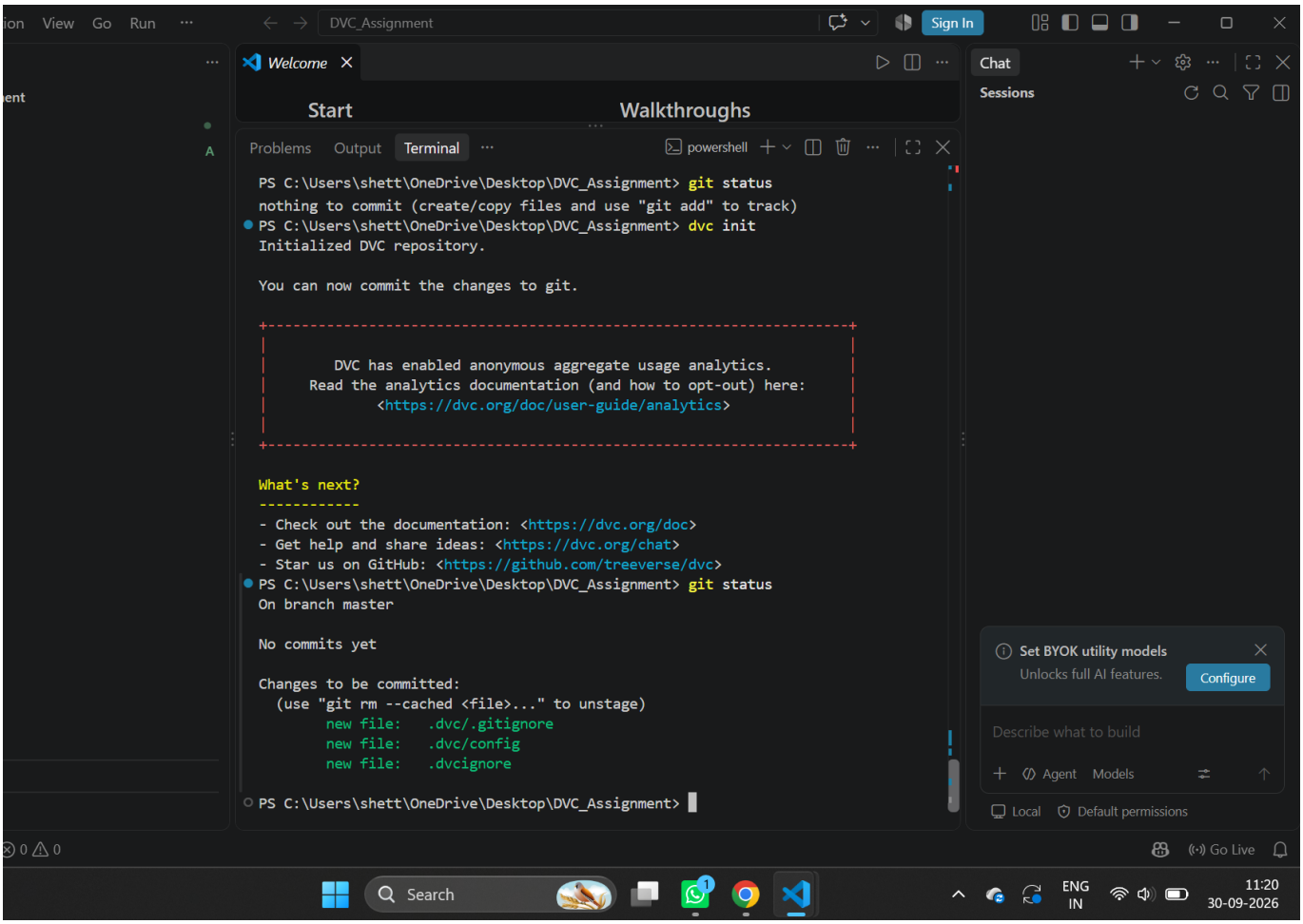
• Git
• DVC
• PowerShell / VS Code Terminal
• GitHub for repository submission

3. Workflow

Initialize → Create Version 1 → dvc add → Git commit + v1.0 → Modify to Version 2 → dvc add → Git commit + v2.0 → git checkout v1.0 → dvc checkout → Verify Version 1

Figure 1 — Git and DVC Initialization

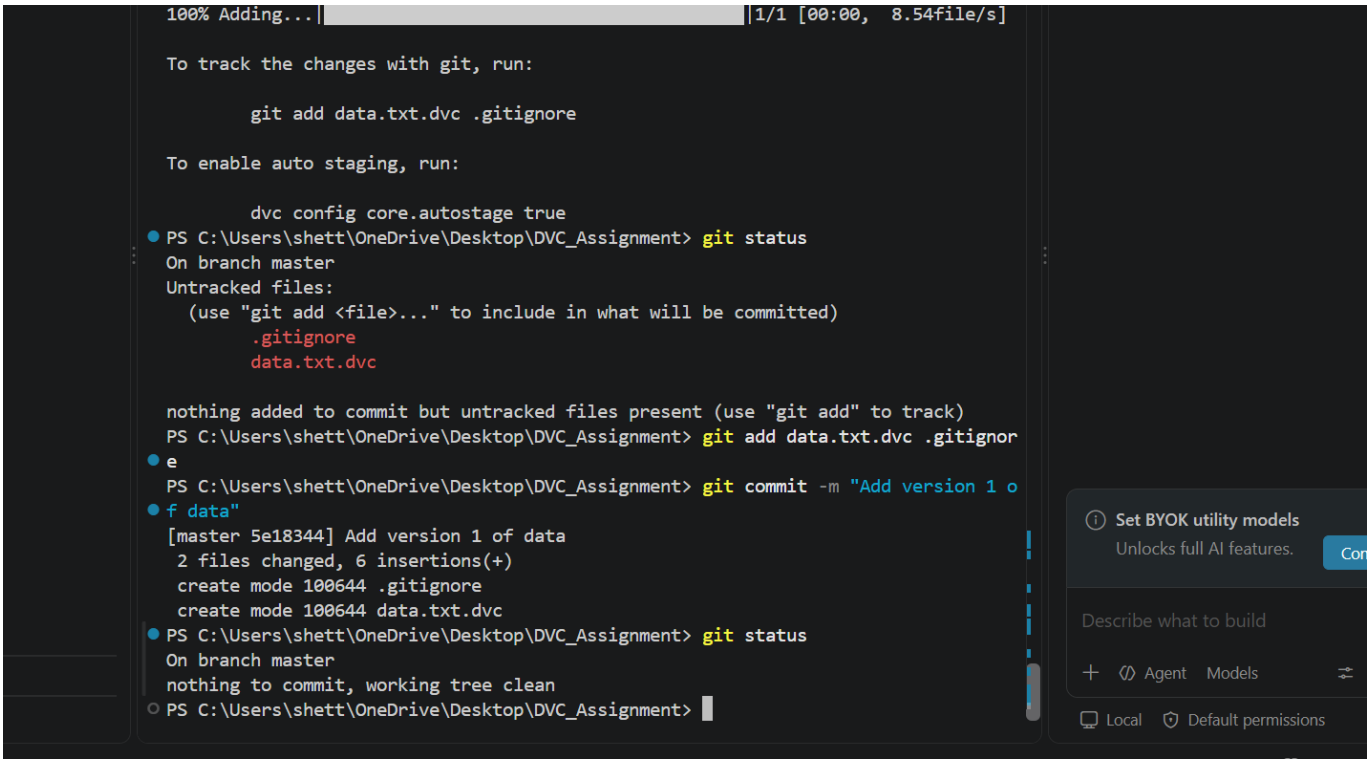
The project repository was initialized with Git using `git init`, followed by DVC initialization using `dvc init`. The DVC configuration files were then prepared for the initial Git commit.



Evidence: 1 — Git and DVC Initialization

Figure 2 — Version 1 of the Dataset

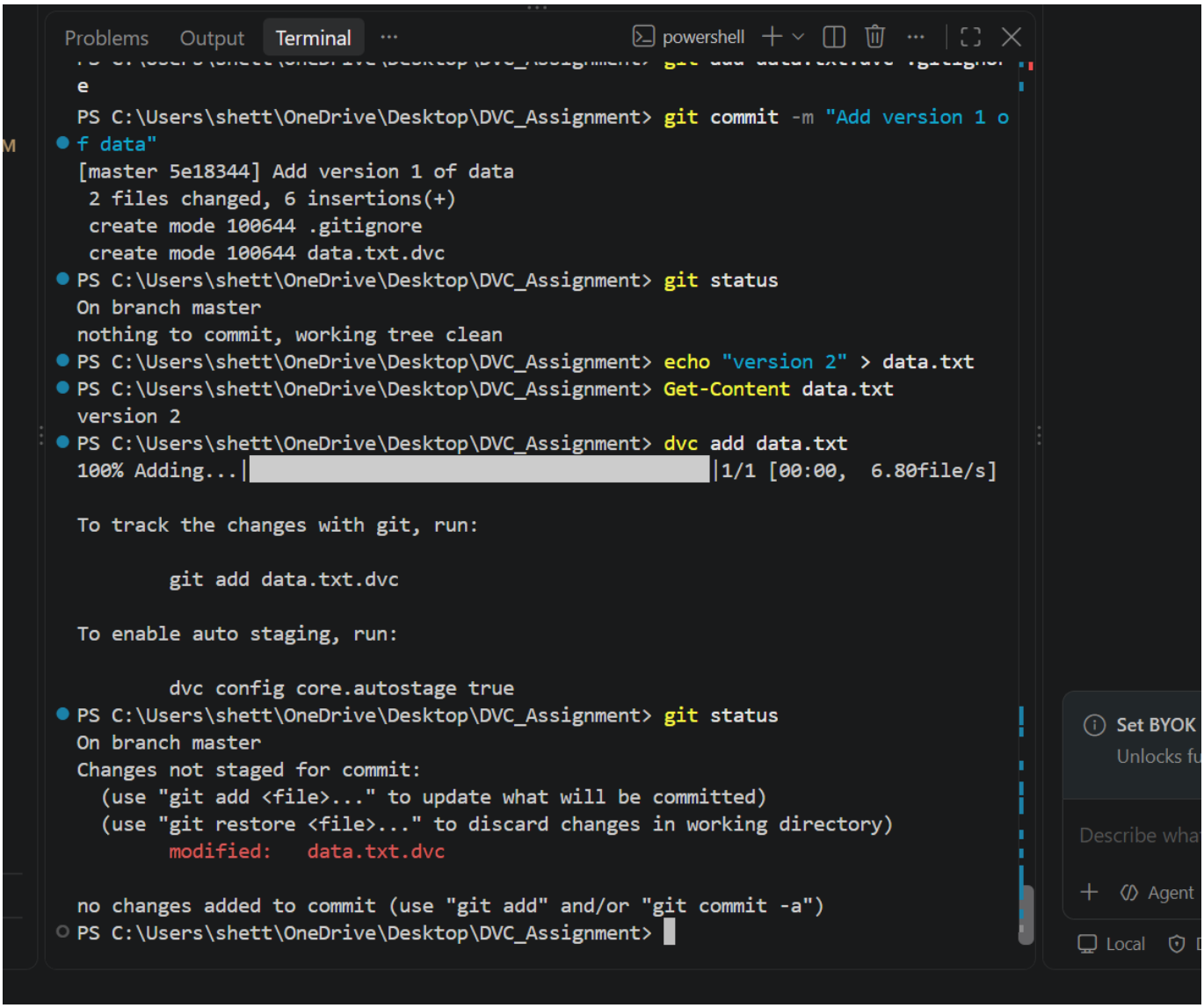
The initial `data.txt` was created with Version 1 content. The file was added to DVC using `dvc add data.txt`, and the generated `data.txt.dvc` metadata was committed to Git.



Evidence: 2 — Version 1 of the Dataset

Figure 3 — Version 2 of the Dataset

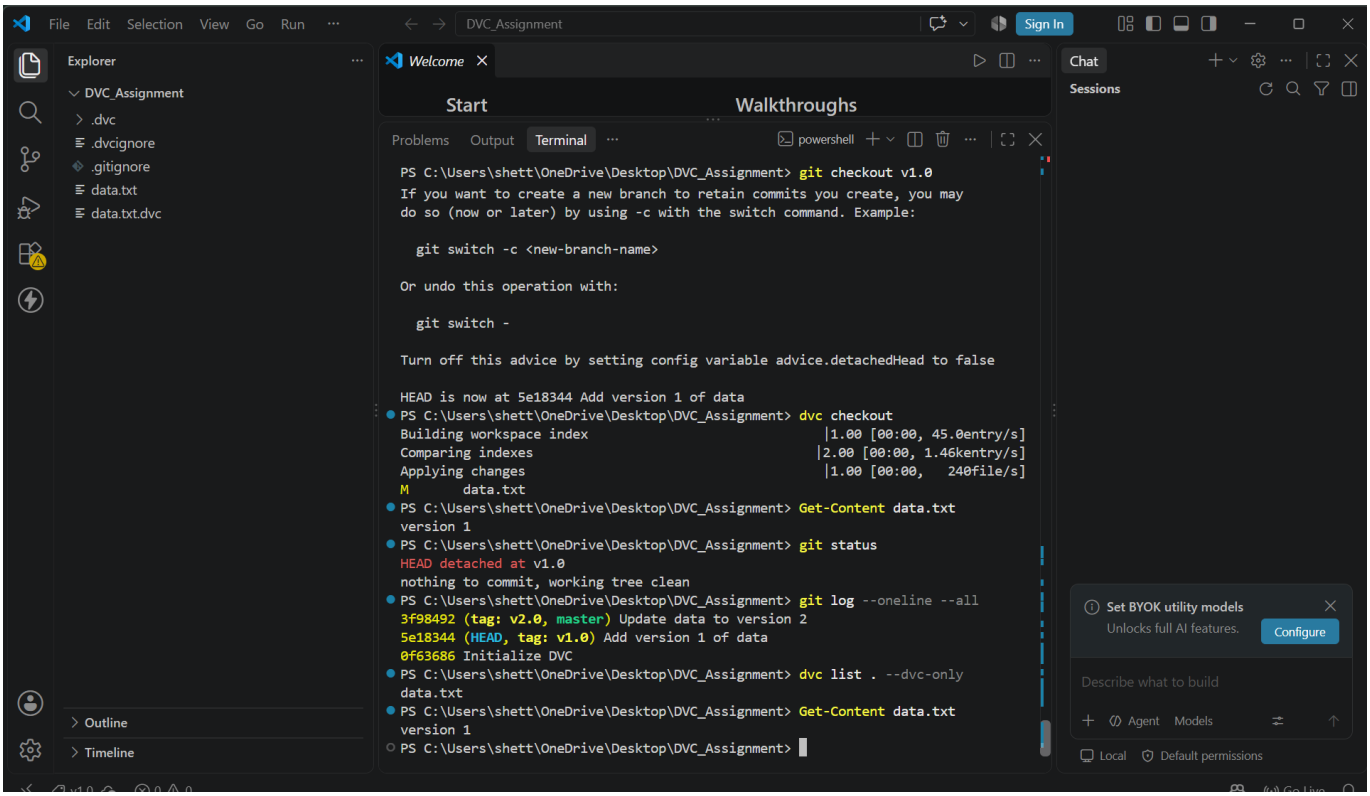
The data was modified from Version 1 to Version 2. Running `dvc add data.txt` updated the DVC metadata, after which the updated metadata was committed to Git.



Evidence: 3 — Version 2 of the Dataset

Figure 4 — Switching Back to Version 1

The Git history contains both `v1.0` and `v2.0`. The project was switched to `v1.0` with Git and then synchronized with `dvc checkout`. The final terminal output verifies that `data.txt` contains Version 1 again.



Evidence: 4 — Switching Back to Version 1

4. Important Commands Used

Stage	Command
Initialize Git	`git init`
Initialize DVC	`dvc init`
Track Version 1	`dvc add data.txt`
Commit Version 1	`git add data.txt.dvc .gitignore  +  git commit -m "Add version 1 of data"`
Tag Version 1	`git tag -a "v1.0" -m "Data version 1.0"`
Track Version 2	`dvc add data.txt`
Commit Version 2	`git add data.txt.dvc  +  git commit -m "Update data to version 2"`
Tag Version 2	`git tag -a "v2.0" -m "Data version 2.0"`
Restore Version 1	`git checkout v1.0`
Synchronize data	`dvc checkout`
Verify data	`Get-Content data.txt`

5. Result

The practical successfully demonstrated two versions of the same data file. The repository contains Git commits and tags for v1.0 and v2.0, while DVC maintains the corresponding data versions. After checking out v1.0 and running dvc checkout, the data file was restored to Version 1.

6. Submission Checklist

• DVC_Assignment project folder
• DVC_Assignment_Final_Report.pdf
• Screen recording of the practical demonstration
• GitHub repository URL
• Key screenshots showing initialization, Version 1, Version 2, and version restoration

Conclusion

The assignment demonstrates how Git and DVC can be used together to maintain reproducible versions of data. Git stores the lightweight DVC metadata and history, while DVC manages the versioned data content.